

SISTEMAS DISTRIBUIDOS

EJERCICIO EVALUABLE 1:

Colas de Mensajes



ID de grupo de prácticas: 81

Nombre de los participantes:

- Ignacio Leal Sánchez
- Mario Cámara Latorre

Correos electrónicos de los participantes:

- 100495680@alumnos.uc3m.es
- 100495734@alumnos.uc3m.es

ÍNDICE

ÍNDICE.....	2
INTRODUCCIÓN.....	2
DISEÑO REALIZADO.....	3
Claves.c.....	3
servidor-mq.c.....	3
proxy-mq.c.....	4
app-cliente(n).c.....	4
COMPILACIÓN Y EJECUCIÓN.....	5

INTRODUCCIÓN

Este proyecto implementa un sistema distribuido basado en colas de mensajes POSIX, permitiendo que múltiples clientes gestionen peticiones para gestionar tuplas de forma concurrente. Utiliza un patrón proxy, donde el cliente envía solicitudes a través de un proxy local (proxy-mq.c), que se comunica con el servidor (servidor-mq.c). El servidor procesa las peticiones y responde mediante colas privadas de cada cliente. El sistema permite insertar, recuperar, modificar, eliminar y verificar tuplas.

DISEÑO REALIZADO

Claves.c

Este archivo es el que tiene las funciones que el servidor usa. Nosotros hemos optado por una **LinkedList** por la simplicidad de estas.

Las funciones que existen son:

- `int set_value(int key, char *value1, int N_value2, double *V_value2, struct Coord value3);`
 - Inserta una nueva tupla en la lista.
- `int get_value(int key, char *value1, int *N_value2, double *V_value2, struct Coord *value3);`
 - Recupera los datos asociados a una clave
- `int modify_value(int key, char *value1, int N_value2, double *V_value2, struct Coord value3);`
 - Modifica una tupla existente.
- `int delete_key(int key);`
 - Elimina una tupla según su clave.
- `int exist(int key);`
 - Verifica si una clave existe en la lista.
- `int destroy();`
 - Elimina todas las tuplas almacenadas.

Dentro de estas funciones antes de acceder o modificar la lista enlazada, la función bloquea el mutex para asegurarse de que solo un hilo pueda acceder a los datos en ese momento, y después desbloquea el mutex. Dentro se suele iterar la **LinkedList** y ejecutar el código necesario.

servidor-mq.c

Aquí se encuentra el código para manejar los mensajes del servidor. Recibe por la cola del servidor las peticiones de los clientes y envía las respuestas por las colas específicas de cada cliente.

Aquí también manda la ejecución de cada función de `claves.c` dependiendo de que se ha indicado en la variable operación en la request.

Este archivo implementa el proxy del lado del servidor, que recibe las peticiones de los clientes a través de colas de mensajes POSIX, las procesa y envía las respuestas a los clientes correspondientes.

Se divide en dos partes principales:

- `tratar_petición()`: función encargada de procesar las peticiones recibidas en la cola del servidor. Lee la estructura de la petición, identifica la operación solicitada y llama a la función correspondiente. Después, abre la cola de respuesta del cliente y envía el resultado de la operación usando `mq_send()`.
- `main()`: configura el servidor y gestiona la recepción de peticiones. Define la cola de mensajes del servidor para asegurar que pueda manejar varias solicitudes al mismo

de manera concurrente. Espera en un bucle infinito peticiones, y cuando las recibe las trata con `tratar_peticion()`.

proxy-mq.c

El archivo contiene la implementación del proxy del lado del cliente, que actúa como intermediario entre la aplicación del cliente y el servidor. Permite que el cliente utilice las funciones sin interactuar directamente con las colas de mensajes.

Cuenta con dos partes principales:

- `send_recv()`: función que gestiona el envío de peticiones al servidor y la recepción de las respuestas por parte de este. Crea una cola privada para recibir la respuesta del servidor, envía la petición a la cola del servidor, espera la respuesta en su cola privada, y una vez cuenta con la respuesta, la almacena en la estructura respuesta y cierra y elimina la cola privada. Se hace uso de colas privadas para cada cliente para recibir respuestas sin posibles errores generados por otras peticiones simultáneas.
- Implementación de `get_value`, `set_value`, `modify_value`, `exist`, `destroy` y `delete_key`: Definen una estructura petición con la operación que se quiere ejecutar, la clave `key` y los datos necesarios. Posteriormente llaman a `send_recv()`, que realiza el envío de la petición y la recepción de la respuesta. Finalmente procesan la respuesta y devuelven el resultado.

app-cliente(n).c

Los diferentes tipos de `app-cliente` demuestran diferentes tests que necesitamos realizar para verificar que el código funcione correctamente.

Cliente1: Es una prueba simple de un `set_value` y un `get_value`. Este primer test evalúa la acción de almacenar y recuperar una clave con sus valores asociados. Se define una clave con valor 11 y se le asignan los siguientes datos: una cadena de caracteres "Ejemplo", un vector de tres elementos {1.1, 2.2, 3.3}, y una estructura con coordenadas {5,6}. El programa primero intenta insertar la clave en el sistema con la función `set_value()`. Si la operación es exitosa, el programa muestra un mensaje indicando que la clave ha sido almacenada correctamente, y en caso contrario genera un error. Una vez insertada la clave, se intenta recuperar con `get_value()`. Si la clave existe, se extraen sus valores y se muestran en pantalla. De lo contrario, genera un error. Con este test se valida que el almacenamiento y la recuperación de claves funcionan correctamente bajo condiciones normales.

Cliente2: El segundo test amplía el primero al incluir más operaciones sobre una misma clave. Se inserta la clave 10 con los mismos valores utilizados en el test anterior y, tras una inserción exitosa, se verifica su recuperación con `get_value()`. A continuación, se modifica la clave con `modify_value()`, cambiando el valor de la cadena de caracteres a "Ejemplo modificado". Si la clave existe, la modificación se lleva a cabo sin problemas y se confirma el cambio con un mensaje en pantalla. Sin embargo, si la clave no existe, el programa notifica el error. Después de modificar la clave, se procede a eliminarla con `delete_key()`. Si la clave ha sido eliminada correctamente, el programa indica el éxito de la operación. En

caso de error, se advierte si la clave ya no existía en el sistema. Para comprobar que la clave ha sido efectivamente eliminada, se usa la función `exist()`. Primero, se verifica que la clave 10 ya no está presente, lo cual debería devolver un 0. Luego, se inserta nuevamente la misma clave y se vuelve a ejecutar `exist()` para confirmar que ahora sí está almacenada, devolviendo un 1. Este test permite evaluar no solo la inserción y recuperación, sino también la modificación, eliminación y verificación de la existencia de una clave en distintos estados.

Cliente3: Es una prueba con 30000 `set_values` es un valor suficientemente elevado para poder probar concurrencia, y que tampoco toma mucho tiempo, en torno a 10-20 segundos. El tercer test está diseñado para evaluar la escalabilidad y el rendimiento del sistema cuando se insertan una gran cantidad de claves de forma consecutiva. En este caso, el programa inserta 30,000 claves en un bucle, comenzando con la clave 10 e incrementándola en cada iteración. Cada clave se almacena con los mismos valores que en los tests anteriores. Mientras este test se ejecuta, es posible evaluar la concurrencia del sistema lanzando otros procesos en paralelo. De esta manera, se puede observar si la inserción masiva de claves afecta el rendimiento o si se producen errores de almacenamiento, y si la concurrencia se cumple de manera adecuada.

Cliente4: El último test evalúa la capacidad del sistema para eliminar todos los datos almacenados y reiniciarse. Primero, se ejecuta la función `destroy()`, que borra todas las claves registradas en el sistema. Si la operación es exitosa, el programa muestra un mensaje confirmando que la base de datos ha sido vaciada correctamente. Después de la destrucción de los datos, se intenta insertar una nueva clave para verificar que el sistema sigue funcionando tras el reinicio. Si `destroy()` ha ejecutado correctamente su función, la base de datos debería estar completamente vacía y la nueva clave debería poder insertarse sin problemas. Este test garantiza que el sistema puede reiniciarse correctamente.

COMPILACIÓN Y EJECUCIÓN

Para llevar a cabo la **compilación** del sistema diseñado se hace uso de un archivo `Makefile` que realiza lo siguiente:

1. Compila el servidor (`servidor`) con `servidor-mq.c` y `claves.c`.
2. Compila el proxy (`proxy-mq.c`) como la biblioteca compartida `libclaves.so`.
3. Compila el cliente (`cliente`) y lo enlaza con `libclaves.so`.
4. Si `make clean` se ejecuta, elimina los archivos generados.

Para llevar a cabo la **ejecución** del sistema, se sigue un orden específico para que el cliente y el servidor se comuniquen correctamente a través de las colas de mensajes POSIX.

1. Iniciar el servidor en una terminal, ya que es quien crea la cola de mensajes y espera solicitudes de los clientes.
2. Ejecutar el cliente en otra terminal. Una vez que el servidor está activo, se puede iniciar uno o varios clientes. Cada cliente enviará peticiones al servidor y esperará una respuesta.
3. Una vez ejecutados los clientes, se puede confirmar la comunicación cliente-servidor a través de los logs de sus respectivas terminales.